WEEK 2 ADD-ON: CODING PRACTICE, GITHUB INTEGRATION & XP SYSTEM

1. OBJECTIVE

Along with programming fundamentals, students will begin solving beginner-friendly coding problems on platforms such as:

- LeetCode
- HackerRank

The objective is to build a regular problem-solving habit and track each student's learning progress through Noxus.

The workflow will be:

Learn Concept

↓

Practice Easy Problems

↓

Write and Understand the Solution

↓

Submit on LeetCode/HackerRank

↓

Sync Solution to GitHub

↓

Add Explanation and README

↓

Track Progress in Noxus

↓

Earn XP

---

2. BEGINNER CODING PRACTICE

Students should begin with very easy problems.

The focus should be on:

- Understanding the problem.
- Identifying input and output.
- Writing the logic.
- Testing the solution.
- Explaining the approach.

The initial difficulty progression should be:

Level 1 — Absolute Beginner

Topics:

- Input and output
- Variables
- Basic arithmetic
- Conditions
- Simple loops

Examples:

- Add two numbers
- Check even or odd
- Find the largest number
- Positive, negative, or zero
- Calculate sum from 1 to N
- Multiplication table
- Factorial
- Simple number reversal

Level 2 — Beginner Logic

Topics:

- Loops
- Conditions
- Basic mathematics
- Strings
- Arrays

Examples:

- Palindrome check
- Count digits

- Sum of digits
- Reverse a string
- Find maximum element
- Find minimum element
- Count vowels
- Linear search
- Calculate array sum

Level 3 — Platform-Based Easy Problems

After students understand the basics, they can begin selected easy problems from:

- LeetCode
- HackerRank

The initial problem set should be curated by mentors and released gradually.

Students should not immediately receive a large list of problems.

Recommended progression:

Day 1: 2 Problems

↓

Day 2: 2–3 Problems

↓

Day 3: 3 Problems

↓

Day 4: Debugging + 2 Problems

↓

Day 5: Challenge + Weekly Review

The focus is consistency and understanding rather than solving the maximum number of questions.

---

3. STUDENT PROBLEM SUBMISSION WORKFLOW

Each student follows this workflow:

1. Open the assigned problem.

2. Read and understand the problem.
3. Identify inputs and outputs.
4. Think about the approach.
5. Write the solution.
6. Test the solution.
7. Submit it on the coding platform.
8. Sync or upload the accepted solution to GitHub.
9. Add a short explanation of their approach.
10. Allow Noxus to track the progress.

---

4. GITHUB REPOSITORY STRUCTURE

Each student can maintain a structured repository.

Example:

```
noxus-coding-journey/
│
├── README.md
│
├── week-02/
│   ├── day-01/
│   │   ├── problem-01/
│   │   │   ├── solution.py
│   │   │   └── README.md
│   │   │
│   │   └── problem-02/
│   │       ├── solution.py
│   │       └── README.md
│   │
│   ├── day-02/
│   └── day-03/
│
└── mini-project/
    ├── source-code/
    └── README.md
```

---

5. README / SOLUTION EXPLANATION FORMAT

Every important problem submission should contain a short explanation.

Example:

Problem

Write the problem name or problem link.

My Understanding

What is the problem asking?

Approach

How did I solve it?

Algorithm

1. Take the input.
2. Process the required values.
3. Apply the condition or loop.
4. Return or print the result.

Complexity

Time Complexity: O(n)

Space Complexity: O(1)

What I Learned

Write one or two points about the concept learned.

This is important because Noxus should evaluate **learning and explanation**, not just accepted solutions.

---

6. LEETHUB-STYLE GITHUB SYNC

Students can be assisted in setting up a browser extension or supported GitHub synchronization workflow that automatically saves accepted coding solutions to their GitHub repository.

The ideal workflow is:

LeetCode / Coding Platform

↓

Accepted Submission

↓

GitHub Sync / Extension

↓

Student Repository Updated

↓

GitHub Commit Created

↓

Noxus Reads Repository Activity

↓

Progress Updated

For the first implementation, Noxus should support a standardized repository structure.

Example:

```
github.com/student/noxus-coding-journey
```

This makes tracking easier.

---

7. GITHUB ACTIVITY TRACKING

Noxus can track relevant repository activity such as:

- Number of coding solutions.
- Commit activity.
- Repository updates.
- README availability.
- Problem folders.
- Weekly consistency.
- Mini-project commits.

The system should focus on activity relevant to the Noxus learning repositories rather than counting every GitHub commit made by the student.

---

8. UNDERSTANDING SCORE

A student's understanding should not be calculated only from the number of solved problems.

Instead, Noxus can create an Understanding Score.

Example:

Understanding Score =

Solution Explanation

- README Quality
- Concept Questions
- Mentor Review
- Weekly Activity

Example scoring:

Component	Weight
Problem Completion	30%
Solution Explanation	25%
Concept Quiz	20%
Consistency	15%
Mentor/Peer Review	10%

This system rewards students who genuinely understand concepts.

---

9. XP SYSTEM

Students earn XP for learning activities.

Basic XP

Activity	XP
Complete a training module	20 XP
Complete assigned task	25 XP
Solve beginner problem	10 XP
Solve intermediate beginner problem	15 XP
Write solution explanation	10 XP
Complete README	5 XP
Pass concept quiz	20 XP
Complete mini-project	100 XP

Activity	XP
Weekly completion	50 XP

---

10. CONSISTENCY BONUS

Students should be rewarded for regular learning.

Example:

3-Day Streak: +15 XP

5-Day Streak: +30 XP

Complete Weekly Tasks: +50 XP

The system should reward consistency rather than only rewarding students who already have advanced programming experience.

---

11. XP PENALTY POLICY

Avoid heavy negative XP penalties.

Instead of removing XP, use:

- Missed task indicators.
- Broken streak.
- Incomplete week status.
- Lower weekly completion percentage.

This keeps the system motivating for beginners.

---

12. STUDENT LEVEL SYSTEM

XP can be converted into levels.

Example:

Level	XP Required
Explorer	0 XP
Learner	100 XP

Level	XP Required
Builder	250 XP
Problem Solver	500 XP
Developer	1,000 XP
Advanced Builder	2,000 XP

The names can later be customized according to the Noxus identity.

---

13. LEADERBOARD SYSTEM

The leaderboard should not rank students only by the number of problems solved.

Instead, use multiple metrics:

XP Score

+

Weekly Consistency

+

Task Completion

+

Understanding Score

+

Project Contribution

Possible leaderboard categories:

- Weekly XP Leaderboard
- Most Consistent Learner
- Problem Solver
- Best Project Contributor
- Most Improved Student

This gives beginners a fair chance to progress.

---

14. WEEKLY CODING DASHBOARD

Students should see:

```
WEEK 2 CODING PROGRESS

Assigned Problems: 12
Completed: 8

GitHub Sync: Connected

README Completed: 6 / 8

Current Streak: 4 Days

Understanding Score: 78%

Week XP: 245 XP

Current Level: Learner
```

---

15. ADMIN AND MENTOR ANALYTICS

Mentors should be able to see:

- Problems assigned.
- Problems completed.
- GitHub repository activity.
- Last active date.
- README completion.
- Explanation quality.
- Quiz performance.
- Weekly consistency.
- XP earned.
- Students who may require support.

Example:

```
Student: ABC

Problems Completed: 8/12
README Completion: 75%
GitHub Activity: Active
Concept Quiz: 70%
Consistency: Good
```

```
Overall Week Progress: 76%

Status: On Track
```

---

16. DATABASE ADDITIONS

The following tables should be added.

coding_platform_profiles

```
id
student_id
platform_name
profile_url
username
is_verified
created_at
```

---

coding_problems

```
id
platform
external_problem_id
title
problem_url
difficulty
topic
week_number
xp_reward
```

---

problem_assignments

```
id
student_id
problem_id
assigned_at
deadline
status
```

---

problem_submissions

```
id
student_id
problem_id
platform
submission_status
github_repository
github_commit
solution_path
submitted_at
```

---

solution_reviews

```
id
submission_id
understanding_score
readme_score
mentor_score
feedback
reviewed_at
```

---

student_xp

```
id
student_id
total_xp
current_level
weekly_xp
updated_at
```

---

xp_transactions

Every XP change should be stored separately.

```
id
student_id
activity_type
reference_id
xp_earned
```

```
description
created_at
```

This makes XP transparent and prevents accidental duplicate rewards.

---

student_streaks

```
id
student_id
current_streak
longest_streak
last_activity_date
updated_at
```

---

17. IMPORTANT FAIRNESS RULE

The system should never assume:

More GitHub commits = More Understanding

Instead:

Verified Completion

+

Solution Explanation

+

Concept Understanding

+

Consistency

+

Mentor Feedback

=

Learning Progress and XP

This will make Noxus much more useful than a simple coding leaderboard.

---

18. FINAL WEEK 2 FLOW

Learn Programming Concept

↓

Practice Beginner Problems

↓

Solve Assigned LeetCode/HackerRank Problems

↓

Submit Accepted Solution

↓

Sync Solution to GitHub

↓

Write README and Explain Approach

↓

Complete Concept Check

↓

GitHub Activity and Learning Progress Tracked

↓

XP Awarded

↓

Weekly Progress Updated

↓

Week 2 Completed

FINAL OUTCOME

At the end of Week 2, every student should have:

- A selected programming track.
- A configured development environment.
- Beginner programming knowledge.
- Experience solving easy coding problems.
- A connected coding platform profile.
- A GitHub coding repository.
- Automatically or manually synced solutions.
- Basic solution documentation.
- An understanding score.
- XP and level progression.
- A visible learning streak.
- A completed mini-project.

The core philosophy is:

> **Don't reward students only for solving more problems. Reward them for learning, understanding, consistency, documentation, and improvement.**